SQL
Employee Management & Analytics Database – Description

```
create database task;

use task;

-- ============================
-- 1      Departments
-- ============================

CREATE TABLE Departments(
dept_id INT PRIMARY KEY,
dept_name VARCHAR(50) NOT NULL UNIQUE);

-- ============================
-- 2      Locations
-- ============================

CREATE TABLE Locations(
location_id INT PRIMARY KEY,
location_name VARCHAR(50) NOT NULL);

-- ============================
-- 3      Job Roles
-- ============================

CREATE TABLE Job_Roles(
role_id INT PRIMARY KEY,
role_name VARCHAR(50) NOT NULL,
min_salary DECIMAL(10,2),
max_salary DECIMAL(10,2));

-- ============================
-- 4      Employee Table
-- ============================

CREATE TABLE Employees(
emp_id INT PRIMARY KEY,
emp_name VARCHAR(100) NOT NULL,
gender VARCHAR(10),
dept_id INT,
role_id INT,
location_id INT,
```

```
join_date DATE,
status VARCHAR(20) DEFAULT 'Active',
FOREIGN KEY (dept_id) REFERENCES Departments(dept_id),
FOREIGN KEY (role_id) REFERENCES Job_Roles(role_id),
FOREIGN KEY (location_id) REFERENCES Locations(location_id));

-- ============================
-- 5      Salaries Table
-- ============================

CREATE TABLE Salaries(
salary_id INT PRIMARY KEY,
emp_id INT,
salary DECIMAL(10,2),
effective_date DATE,
FOREIGN KEY (emp_id) REFERENCES Employees(emp_id));

-- ============================
-- 6      Performance Reviews
-- ============================

CREATE TABLE Performance_Reviews(
review_id INT PRIMARY KEY,
emp_id INT,
review_year INT,
rating DECIMAL(2,1) CHECK (rating BETWEEN 1 AND 5),
FOREIGN KEY (emp_id) REFERENCES Employees(emp_id));

BULK INSERT Locations
FROM 'C:/DATA SET/Locations.csv'
WITH(
FORMAT = 'CSV',
FIRSTROW = 2,
FIELDTERMINATOR = ',',
ROWTERMINATOR = '\n',
TABLOCK);

BULK INSERT Departments
FROM 'C:/DATA SET/Departments.csv'
WITH(
FORMAT = 'CSV',
FIRSTROW = 2,
FIELDTERMINATOR = ',',
ROWTERMINATOR = '\n',
```

```
TABLOCK);

BULK INSERT Employees
FROM 'C:/DATA SET/Employees.csv'
WITH(
FORMAT = 'CSV',
FIRSTROW = 2,
FIELDTERMINATOR = ',',
ROWTERMINATOR = '\n',
TABLOCK);

BULK INSERT Job_Roles
FROM 'C:/DATA SET/Job_Roles.csv'
WITH(
FORMAT = 'CSV',
FIRSTROW = 2,
FIELDTERMINATOR = ',',
ROWTERMINATOR = '\n',
TABLOCK);

BULK INSERT Salaries
FROM 'C:/DATA SET/Salaries.csv'
WITH(
FORMAT = 'CSV',
FIRSTROW = 2,
FIELDTERMINATOR = ',',
ROWTERMINATOR = '\n',
TABLOCK);

BULK INSERT Performance_Reviews
FROM 'C:/DATA SET/Performance_Reviews.csv'
WITH(
FORMAT = 'CSV',
FIRSTROW = 2,
FIELDTERMINATOR = ',',
ROWTERMINATOR = '\n',
TABLOCK);
```

-- Performance Analytics --

-- 1. Top 10 high performing employees

```
SELECT TOP 10 * FROM Performance_Reviews order by rating DESC;
```

	review_id	emp_id	review_year	rating
1	8	108	2026	5.0
2	53	153	2018	5.0
3	15	115	2015	4.9
4	71	171	2019	4.9
5	96	196	2020	4.9
6	34	134	2015	4.8

-- 2. Average rating per department

```
select * from Performance_Reviews
SELECT
   d.dept_name,
   AVG(pr.rating) AS avg_rating
FROM
   Departments d
JOIN
   Employees e ON d.dept_id = e.dept_id
JOIN
   Performance_Reviews pr ON e.emp_id = pr.emp_id
GROUP BY
   D.dept_name
```

	review_id	emp_id	review_year	rating
1	1	101	2019	2.7
2	2	102	2022	3.5
3	3	103	2026	1.7
4	4	104	2016	2.0
5	5	105	2022	1.6
6	6	106	2016	4.0
7	7	107	2023	3.1
8	8	108	2026	5.0

	dept_name	avg_rating
1	Data	2.915789
2	DevOps	2.878260
3	DotNET	2.935000
4	HR	3.094444
5	Java	2.730000

-- 3. employees eligile for promotion(rating>4 & 2+ years) from the above tables

```
SELECT
   e.emp_name,
   d.dept_name,
   pr.rating,
   e.join_date,
```

```
    DATEDIFF(YEAR, e.join_date, GETDATE()) AS years_with_company
FROM Employees e
JOIN Performance_Reviews pr ON e.emp_id = pr.emp_id
JOIN Departments d ON e.dept_id = d.dept_id
WHERE pr.rating > 4.0
  AND DATEDIFF(YEAR, e.join_date, GETDATE()) > 2
```

	emp_name	dept_name	rating	join_date	years_with_company
1	Rohan Gupta	DevOps	5.0	2022-11-28	4
2	Arjun Iyer	Data	4.9	2023-06-24	3
3	Swathi Rao	DotNET	4.5	2017-05-11	9
4	Kavya Sharma	DevOps	4.6	2016-04-05	10
5	Suresh Singh	Java	4.3	2020-10-06	6
6	Naveen Rao	DotNET	4.8	2023-08-23	3
7	Pooja Rao	HR	4.7	2023-02-23	3
8	Vikram Nair	Data	4.2	2022-12-10	4
9	Amit Kumar	DevOps	5.0	2019-01-02	7
10	Meera Kumar	Data	4.8	2017-08-15	9
11	Meera Verma	Java	4.3	2016-12-28	10
12	Rahul Gupta	DevOps	4.8	2017-12-21	9
13	Sneha Kumar	HR	4.9	2017-10-06	9
14	Swathi Rao	DevOps	4.5	2017-12-25	9

-- 4. performance trend over years

```
SELECT
    pr.review_year,
    AVG(pr.rating) AS avg_rating
FROM
    Performance_Reviews pr
GROUP BY
    pr.review_year
ORDER BY
```

-- Salary Analytics --

-- 5. Salary distribution per department

```
SELECT
    d.dept_name,
    COUNT(e.emp_id) AS total_employees,
    MIN(s.salary) AS min_salary,
    MAX(s.salary) AS max_salary,
    ROUND(AVG(s.salary), 2) AS avg_salary
FROM Salaries s
JOIN Employees e
    ON s.emp_id = e.emp_id
```

```
JOIN Departments d

   ON e.dept_id = d.dept_id
GROUP BY d.dept_name
ORDER BY avg_salary DESC;
```

	dept_name	total_employees	min_salary	max_salary	avg_salary
1	DotNET	20	560000.00	980000.00	808000.000000
2	DevOps	23	500000.00	985000.00	752173.910000
3	Java	20	505000.00	990000.00	750500.000000
4	Data	19	515000.00	995000.00	728421.050000
5	HR	18	510000.00	920000.00	691111.110000

-- 6. Highest & lowest salary per role

```
select  role_name, min_salary, max_salary from job_roles;
```

	role_name	min_salary	max_salary
1	HR	40000.00	100000.00
2	SDE 1	100000.00	600000.00
3	Associate	43000.00	115000.00
4	Data Scientist	70000.00	240000.00
5	Manager	50000.00	150000.00

-- 7. Salary vs performance correlation

```
SELECT
   e.emp_name,
   d.dept_name,
   s.salary,
   pr.rating,
   CASE
      WHEN s.salary > (SELECT AVG(salary) FROM Salaries) AND pr.rating < 3.0 THEN
'Overpaid / Underperforming'
      WHEN s.salary < (SELECT AVG(salary) FROM Salaries) AND pr.rating > 4.0 THEN
'Underpaid / High Performer'
      ELSE 'Balanced'
   END AS pay_performance_status
FROM Employees e
JOIN Salaries s ON e.emp_id = s.emp_id
JOIN Performance_Reviews pr ON e.emp_id = pr.emp_id
JOIN Departments d ON e.dept_id = d.dept_id
ORDER BY pr.rating DESC, s.salary DESC;
```

	emp_name	dept_name	salary	rating	pay_performance_status
1	Amit Kumar	DevOps	760000.00	5.0	Balanced
2	Rohan Gupta	DevOps	535000.00	5.0	Underpaid / High Performer
3	Rohan Patel	DotNET	975000.00	4.9	Balanced
4	Sneha Kumar	HR	850000.00	4.9	Balanced
5	Arjun Iyer	Data	570000.00	4.9	Underpaid / High Performer
6	Divya Rao	Data	955000.00	4.8	Balanced
7	Rahul Gupta	DevOps	840000.00	4.8	Balanced
8	Meera Kumar	Data	765000.00	4.8	Balanced
9	Naveen Rao	DotNET	665000.00	4.8	Underpaid / High Performer
10	Pooja Rao	HR	705000.00	4.7	Underpaid / High Performer
11	Kavya Shar...	DevOps	580000.00	4.6	Underpaid / High Performer
12	Neha Rao	DotNET	895000.00	4.5	Balanced
13	Swathi Rao	DevOps	860000.00	4.5	Balanced
14	Swathi Rao	DotNET	575000.00	4.5	Underpaid / High Performer
15	Kiran Sharma	Data	875000.00	4.3	Balanced

-- 8. Employees below department average salary

```
WITH DeptAvg AS (
   SELECT
      e.emp_name,
      d.dept_name,
      s.salary,
      AVG(s.salary) OVER(PARTITION BY d.dept_id) AS dept_avg_salary
   FROM Employees e
   JOIN Salaries s ON e.emp_id = s.emp_id
   JOIN Departments d ON e.dept_id = d.dept_id
)
SELECT
   emp_name,
   dept_name,
   salary,
   dept_avg_salary,
   (dept_avg_salary - salary) AS salary_gap
FROM DeptAvg
WHERE salary < dept_avg_salary
ORDER BY dept_name, salary_gap DESC;
```

	emp_name	dept_name	salary	dept_avg_salary	salary_gap
1	Rohan Kumar	Data	515000.00	728421.052631	213421.052631
2	Manoj Rao	Data	520000.00	728421.052631	208421.052631
3	Divya Reddy	Data	545000.00	728421.052631	183421.052631
4	Arjun Iyer	Data	570000.00	728421.052631	158421.052631
5	Kavya Sharma	Data	590000.00	728421.052631	138421.052631
6	Neha Singh	Data	605000.00	728421.052631	123421.052631
7	Neha Kumar	Data	635000.00	728421.052631	93421.052631
8	Naveen Sha...	Data	655000.00	728421.052631	73421.052631
9	Divya Sharma	Data	695000.00	728421.052631	33421.052631
10	Pooja Gupta	Data	720000.00	728421.052631	8421.052631
11	Arjun Sharma	DevOps	500000.00	752173.913043	252173.913043
12	Rohan Gupta	DevOps	535000.00	752173.913043	217173.913043
13	Divya Rao	DevOps	540000.00	752173.913043	212173.913043
14	Kavya Sharma	DevOps	580000.00	752173.913043	172173.913043
15	Arjun Gupta	DevOps	595000.00	752173.913043	157173.913043

-- Department Growth --

-- 9 Hiring trend per year

```
SELECT
   YEAR(join_date) AS hire_year,
   COUNT(emp_id) AS total_new_hires
FROM Employees
GROUP BY YEAR(join_date)
ORDER BY hire_year ASC;
```

	hire_year	total_new_hires
1	2015	9
2	2016	9
3	2017	15
4	2018	3
5	2019	9
6	2020	13
7	2021	8
8	2022	8
9	2023	11
10	2024	8
11	2025	7

-- 10. Headcount growth by department

```
SELECT
   d.dept_name,
```

```
    COUNT(e.emp_id) AS current_headcount
FROM Departments d
LEFT JOIN Employees e ON d.dept_id = e.dept_id
GROUP BY d.dept_name
ORDER BY current_headcount DESC;
```

	dept_name	current_headcount
1	DevOps	23
2	Java	20
3	DotNET	20
4	Data	19
5	HR	18

-- 11. Attrition rate by department

```
SELECT
    d.dept_name,
    COUNT(e.emp_id) AS total_hired,
    SUM(CASE WHEN e.status != 'Active' THEN 1 ELSE 0 END) AS employees_left,
    CAST(
        (SUM(CASE WHEN e.status != 'Active' THEN 1 ELSE 0 END) * 100.0) /
COUNT(e.emp_id)
        AS DECIMAL(5,2)
    ) AS attrition_rate_percentage
FROM Departments d
LEFT JOIN Employees e ON d.dept_id = e.dept_id
GROUP BY d.dept_name
ORDER BY attrition_rate_percentage DESC;
```

	dept_name	total_hired	employees_left	attrition_rate_percentage
1	HR	18	11	61.11
2	DotNET	20	12	60.00
3	Java	20	11	55.00
4	Data	19	10	52.63
5	DevOps	23	10	43.48

-- Strategic Insights --

-- 12. Employees overdue for promotion

```
SELECT
    e.emp_id,
    e.emp_name,
    e.join_date,
    pr.rating
FROM Employees e
```

```
JOIN Performance_Reviews pr
   ON e.emp_id = pr.emp_id
WHERE DATEDIFF(YEAR, e.join_date, GETDATE()) >= 3
AND pr.rating >= 3;
```

	emp_id	emp_name	join_date	rating
1	102	Kiran Rao	2023-01-02	3.5
2	106	Neha Gupta	2015-10-30	4.0
3	107	Priya Sharma	2019-04-23	3.1
4	108	Rohan Gupta	2022-11-28	5.0
5	109	Divya Rao	2015-11-10	3.6
6	110	Divya Reddy	2019-06-24	3.8
7	114	Amit Singh	2020-06-07	3.8
8	115	Arjun Iyer	2023-06-24	4.9
9	116	Swathi Rao	2017-05-11	4.5
10	117	Kavya Shar...	2016-04-05	4.6
11	118	Suresh Singh	2020-10-06	4.3
12	121	Rahul Reddy	2019-03-13	3.2
13	123	Kavya Reddy	2019-07-22	3.3
14	126	Deepak Sh...	2023-12-13	3.6
15	127	Rohan Red...	2020-01-23	3.5

-- 13. High salary but low performance cases

```
WITH LatestSalary AS (
   SELECT *,
         ROW_NUMBER() OVER (PARTITION BY emp_id ORDER BY effective_date DESC) AS
rn
   FROM Salaries
)
SELECT
   e.emp_id,
   e.emp_name,
   ls.salary,
   pr.rating
FROM Employees e
JOIN LatestSalary ls
   ON e.emp_id = ls.emp_id AND ls.rn = 1
JOIN Performance_Reviews pr
   ON e.emp_id = pr.emp_id
WHERE ls.salary > (SELECT AVG(salary) FROM Salaries)
AND pr.rating <= 2
ORDER BY ls.salary DESC;
```

	emp_id	emp_name	salary	rating
1	197	Naveen Gupta	980000.00	1.2
2	195	Swathi Rao	970000.00	1.6
3	194	Amit Kumar	965000.00	1.5
4	190	Kavya Iyer	945000.00	1.7
5	189	Amit Nair	940000.00	1.4
6	188	Kiran Iyer	935000.00	1.0
7	179	Sneha Singh	890000.00	1.8
8	177	Anita Singh	880000.00	1.2
9	175	Meera Nair	870000.00	1.2
10	163	Suresh Singh	810000.00	1.3
11	162	Manoj Rao	805000.00	1.2
12	160	Suresh Singh	795000.00	1.3
13	155	Anita Reddy	770000.00	1.1
14	152	Swathi Rao	755000.00	1.9

-- 14. Most stable department(lowest attrition)

```
SELECT TOP 1
   d.dept_name,
   COUNT(e.emp_id) AS total_employees,
   SUM(CASE WHEN e.status <> 'Active' THEN 1 ELSE 0 END) AS total_exits,
   ROUND(
      (SUM(CASE WHEN e.status <> 'Active' THEN 1 ELSE 0 END) * 100.0)
      / COUNT(e.emp_id),
   2) AS attrition_rate_percentage
FROM Employees e
JOIN Departments d
   ON e.dept_id = d.dept_id
GROUP BY d.dept_name
ORDER BY attrition_rate_percentage ASC;
```

	dept_name	total_employees	total_exits	attrition_rate_percentage
1	DevOps	23	10	43.480000000000

-- 15. Attendence vs performance analysis

-- 16. location-wise salary comparision

```
SELECT
   l.location_name,
   AVG(s.salary) AS avg_salary
FROM Salaries s
JOIN Employees e
   ON s.emp_id = e.emp_id
```

```
JOIN Locations l
   ON e.location_id = l.location_id
GROUP BY l.location_name;
```

	location_name	avg_salary
1	Hyderabad	743194.444444
2	Pune	730441.176470
3	Trivandrum	772000.000000

-- 17. Gender diversity ratio(if added)

```
SELECT
   gender,
   COUNT(*) AS total_employees,
   ROUND(COUNT(*) * 100.0 / (SELECT COUNT(*) FROM Employees), 2)
      AS percentage
FROM Employees
GROUP BY gender;
```

	gender	total_employees	percentage
1	Female	52	52.000000000000
2	Male	48	48.000000000000

-- 18. Median salary calculation

```
SELECT DISTINCT
   PERCENTILE_CONT(0.5)
   WITHIN GROUP (ORDER BY salary)
   OVER () AS median_salary
FROM Salaries;
```

	median_salary
1	747500

-- 19. Employees with multiple promotion

```
SELECT
   e.emp_id,
   e.emp_name,
   COUNT(s.salary_id) AS promotion_count
FROM Employees e
JOIN Salaries s
   ON e.emp_id = s.emp_id
GROUP BY e.emp_id, e.emp_name
HAVING COUNT(s.salary_id) > 1;
```

```
SELECT emp_id, COUNT(*)
FROM Salaries
GROUP BY emp_id; -- No employee has multiple salary updates –
```

	emp_id	emp_name	promotion_count

	emp_id	(No column name)
1	101	1
2	102	1

-- 20. Identify leadership pipeline candidates

```
SELECT
   e.emp_name,
   d.dept_name,
   jr.role_name AS current_role,
   pr.rating AS last_performance_score,
   DATEDIFF(YEAR, e.join_date, '2026-03-03') AS tenure_years,
   s.salary,
   CASE
      WHEN s.salary >= (jr.max_salary * 0.90) THEN 'Ready for Grade Jump'
      ELSE 'Developmental Lead'
   END AS pipeline_status
FROM Employees e
JOIN Departments d ON e.dept_id = d.dept_id
JOIN Job_Roles jr ON e.role_id = jr.role_id
JOIN Performance_Reviews pr ON e.emp_id = pr.emp_id
JOIN Salaries s ON e.emp_id = s.emp_id
WHERE pr.rating >= 4.5              -- Exceptional performers
  AND e.status = 'Active'
  AND DATEDIFF(YEAR, e.join_date, '2026-03-03') >= 3
ORDER BY pr.rating DESC, tenure_years DESC;
```

	emp_name	dept_name	current_role	last_performance_score	tenure_years	salary	pipeline_status
1	Amit Kumar	DevOps	SDE 1	5.0	7	760000.00	Ready for Grade Jump
2	Rohan Gupta	DevOps	SDE 1	5.0	4	535000.00	Developmental Lead
3	Sneha Kumar	HR	HR	4.9	9	850000.00	Ready for Grade Jump
4	Rahul Gupta	DevOps	HR	4.8	9	840000.00	Ready for Grade Jump
5	Divya Rao	Data	Associate	4.8	7	955000.00	Ready for Grade Jump
6	Naveen Rao	DotNET	Data Scie...	4.8	3	665000.00	Ready for Grade Jump
7	Kavya Shar...	DevOps	Associate	4.6	10	580000.00	Ready for Grade Jump
8	Neha Rao	DotNET	SDE 1	4.5	11	895000.00	Ready for Grade Jump
9	Swathi Rao	DotNET	Data Scie...	4.5	9	575000.00	Ready for Grade Jump
10	Swathi Rao	DevOps	SDE 1	4.5	9	860000.00	Ready for Grade Jump